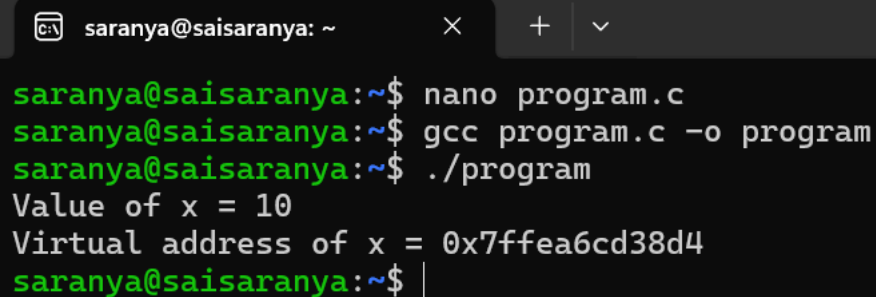


```
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```



A terminal window with a dark background and light green text. The window title bar shows 'saranya@saisaranya: ~' and standard window controls. The terminal displays the following commands and output:

```
saranya@saisaranya:~$ nano program.c
saranya@saisaranya:~$ gcc program.c -o program
saranya@saisaranya:~$ ./program
Value of x = 10
Virtual address of x = 0x7ffea6cd38d4
saranya@saisaranya:~$ |
```

```

#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address      : %p\n", (void *)main);
    printf("Data address       : %p\n", (void *)&global);
    printf("BSS address        : %p\n", (void *)&bss_variable);
    printf("Heap address       : %p\n", (void *)heap);
    printf("Stack address      : %p\n", (void *)&local);

    free(heap);

    return 0;
}

```

```

saranya@saisaranya:~$ nano memory.c
saranya@saisaranya:~$ gcc memory.c -o memory
saranya@saisaranya:~$ ./memory
Code address   : 0x607ac83901a9
Data address   : 0x607ac8393010
BSS address    : 0x607ac8393018
Heap address   : 0x607aeb7f9010
Stack address  : 0x7fffb2687dbc
saranya@saisaranya:~$ |

```

```
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

```
saranya@saisaranya:~$ nano pagetable.c
saranya@saisaranya:~$ gcc pagetable.c -o pagetable
saranya@saisaranya:~$ ./pagetable
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
saranya@saisaranya:~$ |
```

```
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address   = %d\n", virtual_address);
    printf("Page Number       = %d\n", page);
    printf("Offset                = %d\n", offset);
    printf("Frame Number          = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

```
saranya@saisaranya:~$ nano address.c
saranya@saisaranya:~$ gcc address.c -o address
saranya@saisaranya:~$ ./address
Virtual Address = 250
Page Number     = 2
Offset          = 50
Frame Number    = 1
Physical Address = 150
saranya@saisaranya:~$ |
```

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

```
saranya@saisaranya:~$ nano malloc.c
saranya@saisaranya:~$ gcc malloc.c -o malloc
saranya@saisaranya:~$ ./malloc
10 20 30 40 50
saranya@saisaranya:~$ |
```

```
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

```
saranya@saisaranya:~$ nano nullpointer.c
saranya@saisaranya:~$ gcc nullpointer.c -o nullpointer
saranya@saisaranya:~$ ./nullpointer
Before accessing memory
Segmentation fault      ./nullpointer
saranya@saisaranya:~$ |
```